EmbodiedAI Week 6: Visual Servoing

我们这节课的目的，是想要实现类似这样一个效果：



实现这个功能，需要我们回答这几个问题：

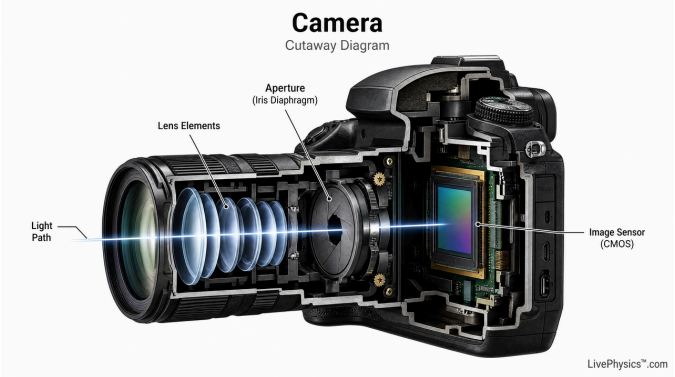
1. 怎么从图中找到我要追踪的目标？
2. 我在图上找到的目标点，它们都来自于三维世界的什么地方？
3. 为了让这些目标点移动到我想让它们待的位置（例如画面中央），相机应该如何运动？
4. 为了让相机朝这个方向运动，我的机械臂的每个关节应该怎么运动？

一、相机成像

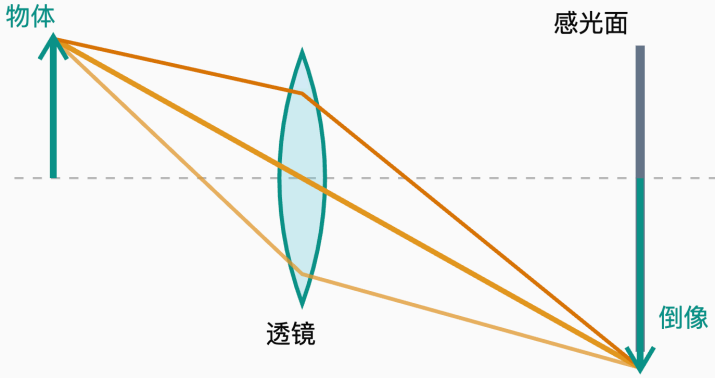
> 📷 三维物理世界中的一个点，是如何被投影成二维像素的？

1.1 当我拍照的时候，相机里面发生了什么？

我们平时用的相机通常是这样的结构：



相机内部构造



透镜组可以简化为一个凸透镜，用来汇聚光线（在对焦时）

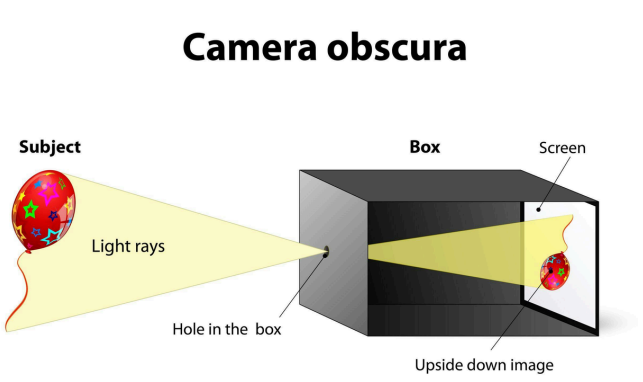
它包含三个部分：

1. **透镜组**：负责收集光线。
   它的功能可以用一个理想凸透镜来近似理解：将来自同一个物点、沿不同方向传播的光线折射，使折射后的光线在镜头后方汇聚。当相机正确对焦时，这个汇聚点就在 CMOS 感光面上，从而形成清晰的图像。
2. **快门**：负责控制相机的曝光时间。
   拍照的一瞬间快门打开，投射到 CMOS sensor 上的光线开始被转换成电信号。一段时间后快门自动关闭，曝光结束。快门包括机械快门（可以理解成物理上的挡板）和电子快门（直接通过电路控制 CMOS 曝光时间）。
3. **感光元件**（通常是 CMOS sensor）：负责把光信号转换为电信号。
   它是一个二维感光像素阵列，每个像素中的光电二极管将接收到的光转换为电荷，并在曝光期间不断积累。曝光结束后，这些电荷通过读出电路转换为电压，再经过模数转换（ADC）、图像处理等流程，就形成了照片。

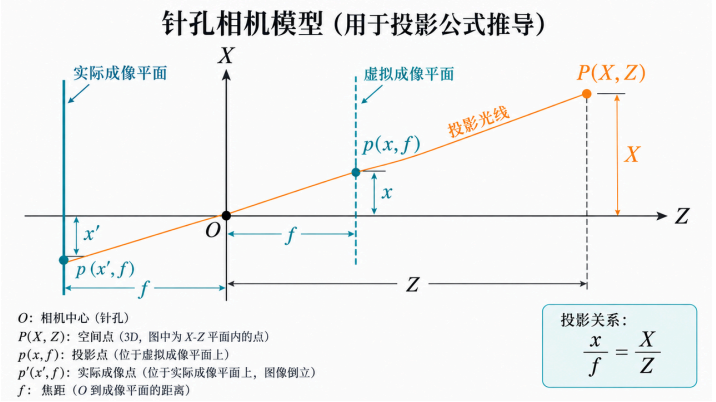
1.2 小孔成像

我们关心的问题是：**三维世界中的一个点，被投影到了像平面上的哪个位置？**

由于经过凸透镜光心的光线可以近似看作沿直线传播、不发生偏折，因此对于我们关心的这个问题，可以进一步把透镜系统等效成小孔成像。其中，穿过小孔、与成像平面垂直的直线我们称为“光轴”：



这就是“针孔相机模型”



针孔相机模型公式推导示意图

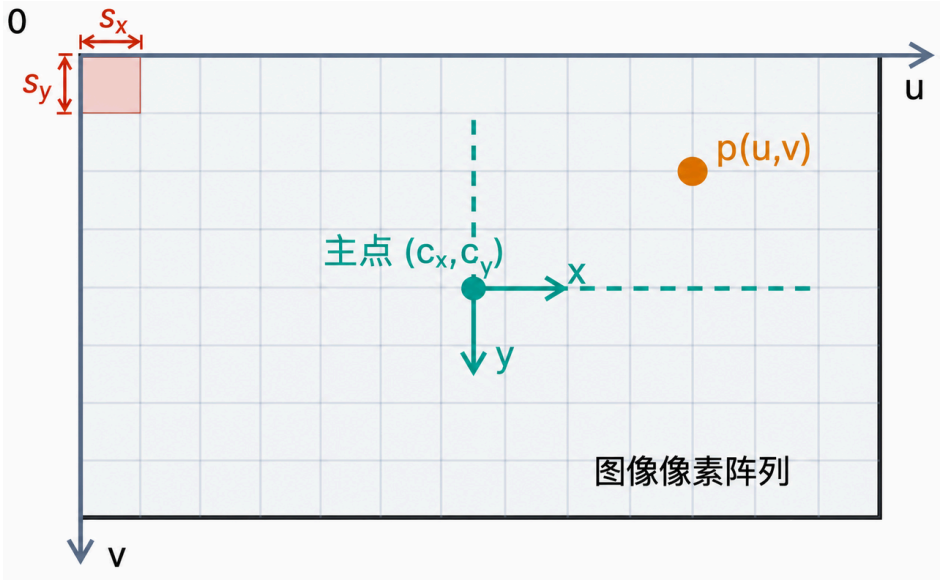
在这个二维的例子中，我们可以通过相似三角形得到：

$$\frac{x}{f} = \frac{X}{Z}$$

同理，对于成像平面的另一个维度，也有：

$$\frac{y}{f} = \frac{Y}{Z}$$

这里的 x 和 y 都是相对于光轴与成像平面的交点（称为“主点”）来定义的。通常 OpenCV 库会把图像二维坐标系放在左上角，像下面这样：



其中，(u, v) 是这个点在图像上的像素坐标。假设 CMOS 每个像素的宽度为 s_x 毫米，高度为 s_y 毫米，那么：

$$u = \frac{x}{s_x} + c_x = \frac{f}{s_x}\frac{X}{Z} + c_x = f_x\frac{X}{Z} + c_x$$

$$v = \frac{y}{s_y} + c_y = \frac{f}{s_y}\frac{Y}{Z} + c_y = f_y\frac{Y}{Z} + c_y$$

我们可以把它写成矩阵形式：

$$Z\begin{bmatrix}u\\v\\1\end{bmatrix}=\underbrace{\begin{bmatrix}f_x&0&c_x\\0&f_y&c_y\\0&0&1\end{bmatrix}}_{K}\begin{bmatrix}X\\Y\\Z\end{bmatrix}$$

其中 K 我们就称为相机的**内参矩阵**。当一个相机被生产出来的时候，它的内参矩阵就固定了。只要知道了这个矩阵，给我任何一个三维空间中的坐标 (X,Y,Z)，我就能知道它被投影到图像中的哪个像素点 (u,v)。

但是反过来，对于图像中的点 (u,v)，即便知道相机内参，我也只能确定 (X,Y,Z) 在三维空间中的哪一条射线上，而不能直接确定确切的三维坐标点。

1.3 相机内参标定

我手上已经有了一台相机，我怎么测算出它的内参矩阵?

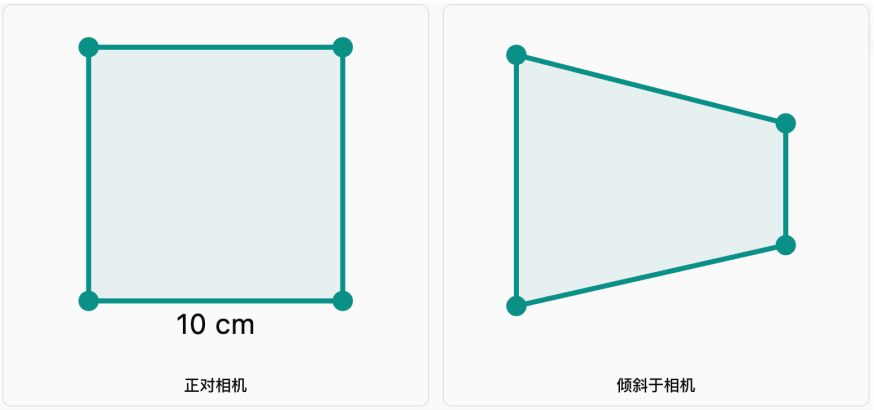
具体来说，我们要求解的是 f_x，f_y，c_x，c_y；假设我已知两个点的三维坐标和它们投影到图像上的像素坐标，即 $P_1=(X_1,Y_1,Z_1)\to(u_1,v_1)$、$P_2=(X_2,Y_2,Z_2)\to(u_2,v_2)$，那么我可以直接根据上一节的推导列出方程组：

$$\begin{cases}u_1=f_x\frac{X_1}{Z_1}+c_x\\v_1=f_y\frac{Y_1}{Z_1}+c_y\\u_2=f_x\frac{X_2}{Z_2}+c_x\\v_2=f_y\frac{Y_2}{Z_2}+c_y\end{cases}$$

但是问题是，通常来说我没法同时知道一个点的三维坐标和像素坐标。

因此另一种方法就是，假设我们已知一些点的相对关系，比如我们在纸上画一个正方形，我们已知正方形边长为 10cm，那么在正方形自身的坐标系下，四个点坐标分别为：

$$P_0=(0,0,0),P_1=(10,0,0),P_2=(0,10,0),P_3=(10,10,0)。$$



正方形摆放在不同姿态时，四个角点投影到像平面的位置也不同

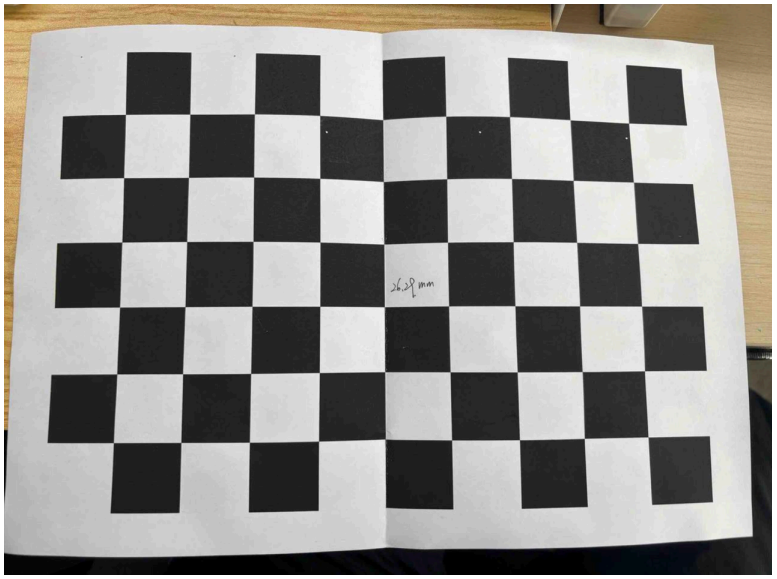
我们不知道正方形坐标系相对于相机坐标系的位姿（包含 3 维相对位置 + 3 维相对朝向），但我们知道拍的每张照片，都对应了一个边长 10 cm 的正方形在某个位姿下，经过同一套相机内参形成的投影。因此我们可以把相机内参的计算写成下面这个优化问题：

$$\underset{f_x,f_y,c_x,c_y,\{R_j,t_j\}}{\arg\min} \mathcal{L} = \sum_{j=1}^{M}\sum_{i=1}^{N} \left\| \begin{bmatrix} f_x \frac{X_{ij}}{Z_{ij}} + c_x \\ f_y \frac{Y_{ij}}{Z_{ij}} + c_y \end{bmatrix} - \begin{bmatrix} u_{ij} \\ v_{ij} \end{bmatrix} \right\|_2^2, \quad \begin{bmatrix} X_{ij} \\ Y_{ij} \\ Z_{ij} \end{bmatrix} = R_j P_i + t_j$$

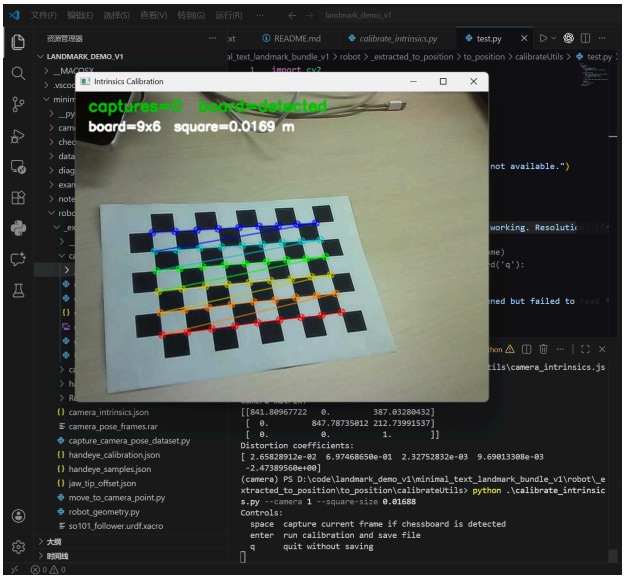
其中 $R_j \in \mathbb{R}^{3\times3}$ 和 $t_j \in \mathbb{R}^3$ 分别为拍第 j 张照片时，正方形坐标系相对于相机坐标系的旋转和平移。

这里的 $\mathcal{L}$ 我们叫**重投影误差**，也就是根据估计出的相机内参和正方形坐标系位姿，能多好地解释每张图片中正方形角点的像素坐标。它可以用来度量估计出的相机内参的准确性。

实际做相机内参标定的时候，我们都会用下面这样的棋盘格纸（相当于几十个正方形），让相机从不同角度和距离拍这张纸，拟合一个内参矩阵，使其能解释所有图像中棋盘格格点在图像上的像素坐标。



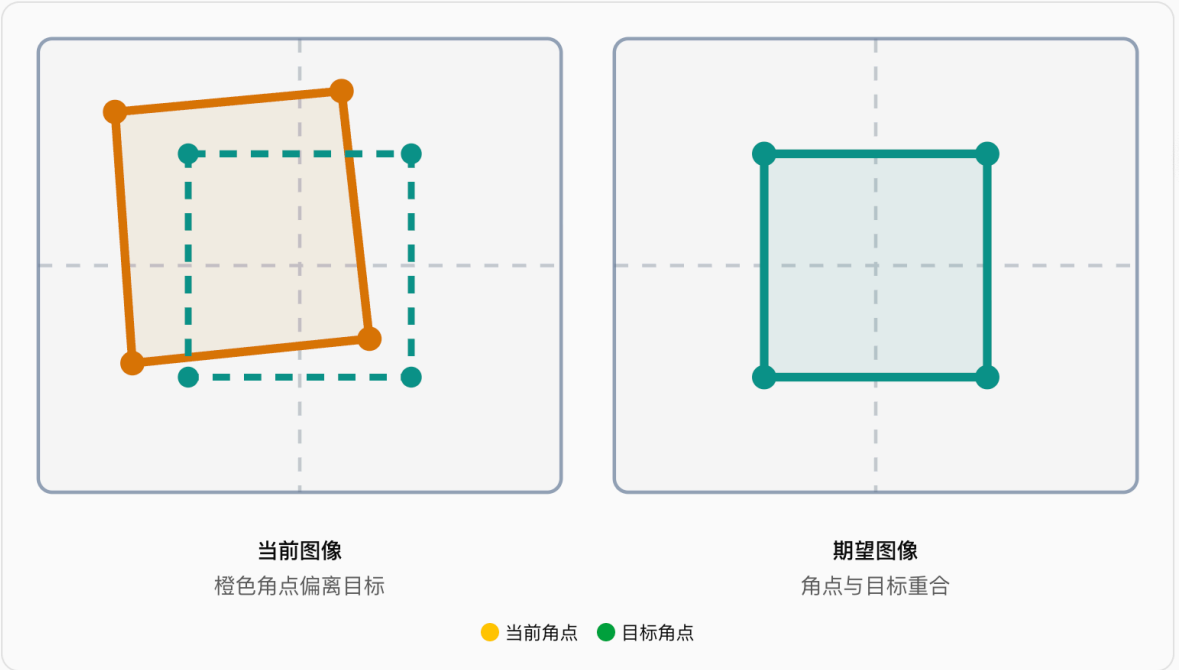
棋盘格点边长 25 mm



OpenCV 有成熟的算法可以检测每个格点的像素坐标

二、视觉伺服

🤔 为了让当前画面里看到的几个“点”出现在各自的目标位置，我的相机应该怎么运动？



假设正方形当前在相机画面中位于上图橙色框的位置，而我们的目标是计算相机如何移动，使得正方形能够到达绿色框的位置。为了解这个问题，我们需要先推导一下相机运动如何影响画面中点的运动。

2.1 相机平移对像素坐标的影响

假设相机平移的速度为：

$$\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}$$

那么三维点在相机坐标系下的坐标的移动速度就等于 $-\mathbf{v}$，即 $\dot{X} = -v_x, \dot{Y} = -v_y, \dot{Z} = -v_z$。

代入 $u = f_x \frac{X}{Z} + c_x$ 和 $v = f_y \frac{Y}{Z} + c_y$，可以得到

$$\dot{u} = -\frac{f_x}{Z} v_x + \frac{f_x X}{Z^2} v_z$$
$$\dot{v} = -\frac{f_y}{Z} v_y + \frac{f_y Y}{Z^2} v_z$$

到这里我们算出来相机平移对于点 (X, Y, Z) 在画面上的像素坐标 (u, v) 的影响。

2.2 相机旋转对像素坐标的影响

假设相机旋转的角速度为：

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

对于一个在世界中静止的点 $P=(X,Y,Z)$，它在相机坐标系中的位置变化速度为 $\dot{P}=-\boldsymbol{\omega}\times P$，这里“$\times$”表示叉乘。我们把叉乘展开，可以得到：

$$\begin{aligned}\dot{X}&=-v_x-\omega_y Z+\omega_z Y\\ \dot{Y}&=-v_y-\omega_z X+\omega_x Z\\ \dot{Z}&=-v_z-\omega_x Y+\omega_y X\end{aligned}$$

接着用上一节一样的方式，我们就可以求出相机旋转作用下的 $\dot{u}$ 和 $\dot{v}$ 。

合并在一起，假设相机运动为：

$$\mathbf{v}_c=\begin{bmatrix}v_x\\v_y\\v_z\\\omega_x\\\omega_y\\\omega_z\end{bmatrix}$$

把上面的计算结果合起来，我们得到了：

$$\begin{bmatrix}\dot{u}\\\dot{v}\end{bmatrix}=\underbrace{\begin{bmatrix}-\frac{f_x}{Z}&0&\frac{f_x\alpha}{Z}&f_x\alpha\beta&-f_x(1+\alpha^2)&f_x\beta\\0&-\frac{f_y}{Z}&\frac{f_y\beta}{Z}&f_y(1+\beta^2)&-f_y\alpha\beta&-f_y\alpha\end{bmatrix}}_{L_i}\mathbf{v}_c,\quad \alpha=\frac{X}{Z},\beta=\frac{Y}{Z}$$

矩阵 L_i 描述了相机速度对像素坐标速度的影响，我们称之为**图像交互矩阵（Interaction Matrix）**。

我们看到，图像交互矩阵里面有一个我们不知道的变量 Z，也就是点 P 的深度。但由于我们实际有四个点，且我们知道这四个点构成一个正方形 & 正方形的边长已知 & 相机内参已知；那么根据之前我们求解内参矩阵的流程，我们可以算出正方形在相机坐标系下的位姿，进而算出每个点的 (X,Y,Z) 值。

前面的计算只考虑一个点，对于正方形的四个角点，直接把每个角点的 (u,v) 坐标和 L_i 矩阵拼接即可。 即：

$$\mathbf{s}=\begin{bmatrix}u_1\\v_1\\u_2\\v_2\\u_3\\v_3\\u_4\\v_4\end{bmatrix},\ L=\begin{bmatrix}L_1\\L_2\\L_3\\L_4\end{bmatrix},\ \underbrace{\dot{\mathbf{s}}}_{8\times1}=\underbrace{L}_{8\times6}\underbrace{\mathbf{v}_c}_{6\times1}$$

2.3 求解视觉伺服

我们定义我们最终目标是为了让 $e = \mathbf{s} - \mathbf{s}^*$ ，即当前观察到的四个点的位置和它们目标位置之差。我们的目的是：

1. 随着运动的进行，让 e 逐步减小，最后等于 0。
2. 在 e 很大时，用更快的相机移动速度，在 e 很小时，放慢移动速度。

出于这两个目的，我们希望：

$$\dot{e} = -\lambda e$$

也就是：误差越大，误差向 0 变化的速度越快；误差等于 0 时，稳定保持不变。 λ 控制了误差变化的总体速度。

我们代入 $\dot{\mathbf{e}} = \dot{\mathbf{s}} = L\mathbf{v}_c$ ，得到：

$$L\mathbf{v}_c = -\lambda \mathbf{e}$$

我们前面已经求解得出了 L ，我们从照片上也知道了当前时刻的 e ，那我们只需要计算：

$$\mathbf{v}_c^* = \arg\min_{\mathbf{v}_c} \|L\mathbf{v}_c + \lambda \mathbf{e}\|_2^2$$

这是一个最小二乘优化，可以直接求解。

至此我们计算出了让当前角点靠近目标角点，相机所需要的速度。下一个问题就是：为了让相机产生这样一个速度（或者说单位时间的移动），我的关节角应该怎么变？它可以拆成两步：

1. 为了让相机产生速度 $\mathbf{v}_c^*$ （或者说在下一个 Δt 内运动 $\mathbf{v}_c^* \Delta t$ ），机械臂末端在下一个时间步应该移动到哪里？—— 这是我们即将要讲的手眼标定
2. 为了让机械臂末端在下一个时间步移动到目标位置，我的关节角应该怎么变？—— 这是我们上节课说的 IK。

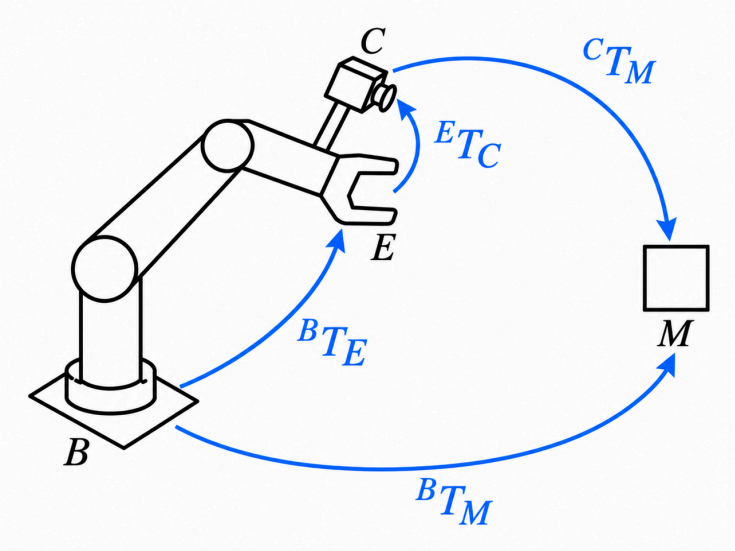
> 注：我们这里讲的视觉伺服是基于图像的视觉伺服（Image-Based Visual Servoing，IBVS），它的特点是完全根据像素坐标来定义误差、并进行反馈控制。
>
> 与之相应地，还有基于位置的视觉伺服（Position-Based Visual Servoing，PBVS），它会先估计目标与相机之间的三维相对位姿，再在三维空间中定义和减小误差。

三、手眼标定

🦾 相机和机械臂末端之间的相对位置是什么？

我们知道，对于我们的机械臂而言，机械臂末端和相机之间的相对位置是固定的（除非固定摄像头的螺丝松了）。意味着它们之间存在一个固定的 3 维平移 + 3 维旋转的关系。

我们通常用 ${}^{E}T_{C}$ 来表示这个关系，表示在末端执行器（**E**nd **E**ffector）坐标系下，相机（**C**amera）的位姿。它是一个 4*4 大小的矩阵，右乘这个矩阵可以将相机坐标系中的坐标转换成末端执行器坐标系下的坐标。



和之前一样，假如我们在环境中放一个正方形，记为 M ，那么有：

$$ {}^{B}T_{M} = {}^{B}T_{E}\ {}^{E}T_{C}\ {}^{C}T_{M} $$

假设我让机械臂运动到某一组关节角 q_1 ，那么就有：

$$ {}^{B}T_{M} = {}^{B}T_{E}^{(1)}{}^{E}T_{C}{}^{C}T_{M}^{(1)} $$

然后让机械臂运动到另一组关节角 q_2 ，同理有：

$$ {}^{B}T_{M} = {}^{B}T_{E}^{(2)}{}^{E}T_{C}{}^{C}T_{M}^{(2)} $$

我们知道这个过程中只有机械臂的姿态在变化，正方形相对于机械臂基座的位置是不变的，因此等式左边可以消掉：

$$ {}^{B}T_{E}^{(1)}{}^{E}T_{C}{}^{C}T_{M}^{(1)} = {}^{B}T_{E}^{(2)}{}^{E}T_{C}{}^{C}T_{M}^{(2)} $$

其中：

- ${}^{B}T_{E}^{(1)}$ 、 ${}^{B}T_{E}^{(2)}$ 我们分别可以用 q_1 和 q_2 加上 FK 求出来；
- ${}^{C}T_{M}^{(1)}$ 、 ${}^{C}T_{M}^{(2)}$ 在我们知道“正方形”这个形状以及它的边长、加上相机内参 K 时，也可以求出来。

理论上，最少三个姿态，我们就能够求出手眼矩阵 ${}^{E}T_{C}$ 。

四、课堂练习 & 课后作业

1. **相机标定**：使用 OpenCV 自带的相机标定工具和棋盘格纸完成。步骤如下：
    a. 改变摄像头和棋盘格纸之间的相对位置，每改变一次就拍一张照。为了准确，建议拍 15 张及以上。

b. 使用 OpenCV 库完成相机内参标定，并保存计算得出的相机内参。

c. **要求：**重投影误差在 1.0 左右。棋盘格纸边长为 25mm。

2. **手眼标定**：

a. 固定机械臂基座和标定板的位置，只通过发送关节角指令改变机械臂的姿态。每改变一次姿态，就拍一张照片（照片中，棋盘格需要完整可见）。为了准确，建议拍 12 张及以上。

b. 基于 OpenCV 库和你们之前已经写过的运动控制与 FK，完成手眼矩阵的计算，保存手眼标定结果，截图并解释相机和末端位姿之间的相对位置（如果算出来相机离末端几米远，想必你也不好意思解释它）。

c. 估计出的标定板坐标原点的位置和实际位置各个坐标轴的误差最好在 5 cm 以内。

3. **视觉伺服**：从以下两种方案中选择一种实现 ——

a. 简单版：直接按照固定运动规律，例如 ArUco 码的中心点如果落在图像左侧/右侧/上方/下方 40% 宽度的位置，那就让相机往左侧/右侧/上方/下方移动 1cm；如果落在图像中间区域，就让相机往前移动 1cm。

b. 进阶版：根据上面第二章的推导过程，实现让 ArUco 码的四个角点落在画面中央四个目标位置的视觉伺服。

4. **加分项**：从 ArUco 扩展到任何物体（例如通过mediapipe追踪手、通过foundation model+光流追踪tracking任何物体）